Milestone 2: Process Orders

Build-a-Store Semester Project

GOAL	FOCUS	GRADING
Extend the store with orders and linear ADTs	Testing, Big-O, Stack, Queue, LinkedList	70 auto + 30 manual

1. Project Overview

Milestone 1 established the object-oriented foundation of the online store. In Milestone 2, the store begins processing real orders. You will add an Order class and build two linear abstract data types: a Queue for orders waiting to be processed and a Stack for processed-order history. Both ADTs will be implemented using the LinkedList you create.

This milestone also puts Modules 3 and 4 into practice. You will write unit tests with Python's built-in unittest framework, test behavior rather than implementation details, and describe the worst-case running time of important operations using Big-O notation.

Keep it manageable. This is an extension of Milestone 1, not a new project. Keep the Product, Customer, and ShoppingCart behavior that already works. Add the required functionality and connect the pieces.

2. What Stays the Same and What Changes

From Milestone 1	Milestone 2
Product, Customer, ShoppingCart, and Store remain the core classes.	Order, Node, LinkedList, Stack, and OrderQueue are added.
Store still uses Python lists for products and customers.	Store now also manages orders, a waiting-order queue, and processed-order history.
ShoppingCart still stores Product objects in a Python list.	ShoppingCart gains clear() so checkout can empty the cart.
Existing M1 method names and behavior stay the same.	Store gains order methods, checkout, and order processing.

3. The New Classes

Class	Purpose
Order	Represents a completed checkout. It keeps the customer, purchased products, and current status.
Node	Stores one item and a reference to the next node.
LinkedList	Stores items as connected nodes using head and tail references.
Stack	Implements LIFO behavior by wrapping a LinkedList.
OrderQueue	Implements FIFO Queue behavior by wrapping a LinkedList.

4. How the Classes Work Together

Checkout converts the contents of a customer's ShoppingCart into an Order. The Store keeps the order in its permanent orders list and enqueues it in the waiting Queue. When an order is processed, the oldest order leaves the Queue and is pushed onto the Stack that stores processed-order history.

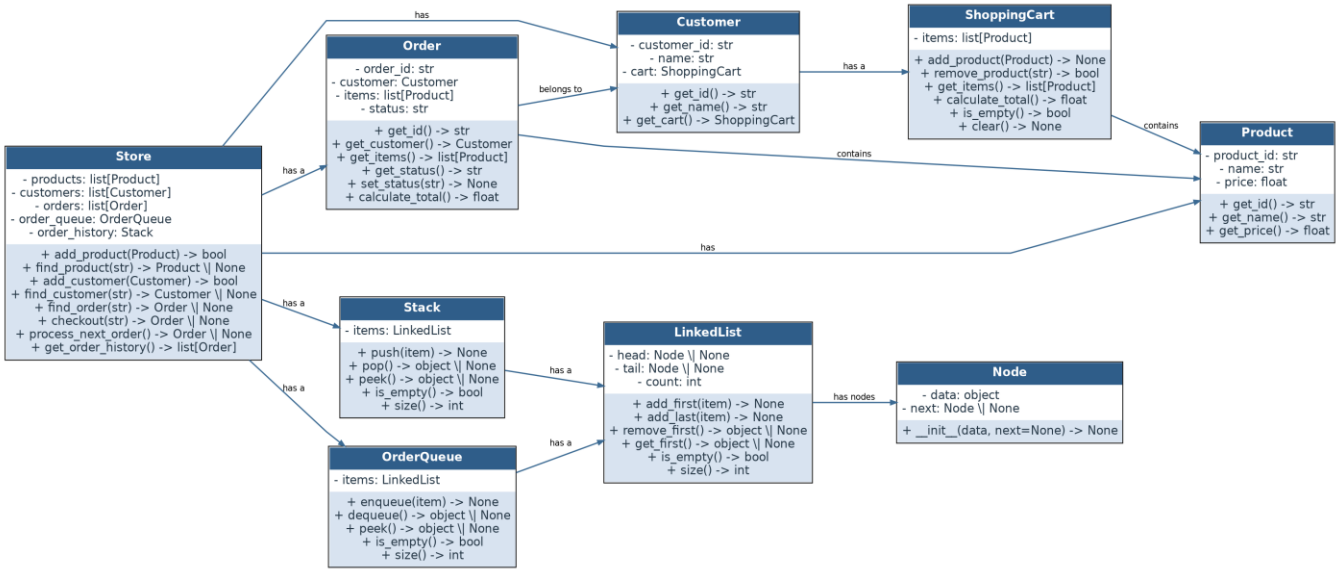


Figure 1. Milestone 2 UML class diagram

- Customer has a ShoppingCart; ShoppingCart contains Product objects.
- Order belongs to a Customer and contains its own copy of Product objects.
- Store has Products, Customers, Orders, an OrderQueue, and a Stack for history.
- Stack and OrderQueue each have a LinkedList. This is composition and the wrapper pattern.
- LinkedList has Node objects connected through next references.

5. Required Classes and Methods

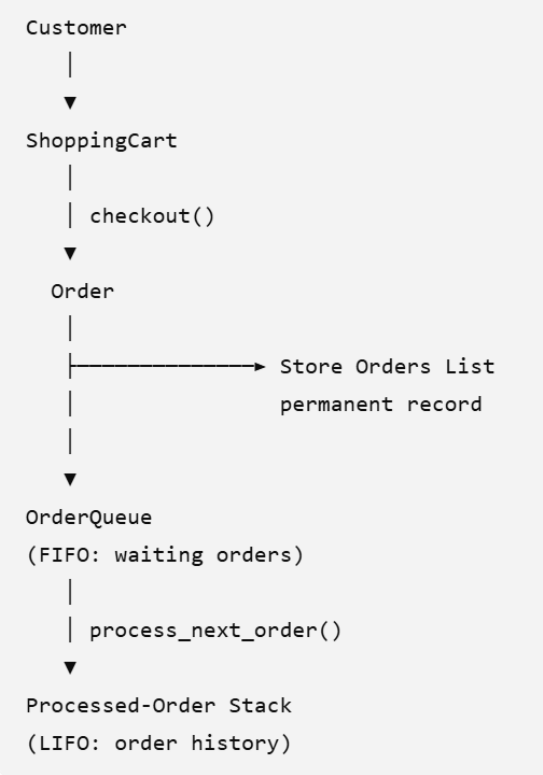
Follow the class names, method names, parameters, and return types exactly. You may add helper methods, but do not rename or change the required methods.

5.1 Order

An Order stores a snapshot of the customer's purchased products. If the customer changes their cart after checkout, the existing Order must not change.

Method	Purpose	Return type
__init__(order_id: str, customer: Customer, items: list[Product])	Create an order with the given ID, customer, and purchased items. The initial status is PENDING.	None
get_id()	Return the order ID.	str
get_customer()	Return the Customer who placed the order.	Customer
get_items()	Return the products in the order.	list[Product]
get_status()	Return the current status.	str
set_status(status: str)	Change the status. Valid values are PENDING, PROCESSING, and COMPLETED.	None
calculate_total()	Return the total price of all products in the order.	float

The OrderQueue represents orders waiting to be processed and follows FIFO (First In, First Out) behavior. When an order is processed, it is removed from the queue and added to a processed-order Stack, which follows LIFO (Last In, First Out) behavior and provides the store's order history.



5.2 Node and LinkedList

Do not use a Python list to store the elements of LinkedList. Each Node stores one item and a reference to the next Node. Keep both head and tail references so add_first() and add_last() are O(1).

Method	Purpose	Return type
Node.__init__(data, next=None)	Create a node that stores data and a reference to the next node.	None
LinkedList.__init__()	Create an empty linked list with head, tail, and a size counter.	None
add_first(item)	Add an item to the beginning of the linked list.	None
add_last(item)	Add an item to the end of the linked list.	None
remove_first()	Remove and return the first item. Return None if empty.	object | None
get_first()	Return the first item without removing it. Return None if empty.	object | None
is_empty()	Return True when the linked list is empty.	bool
size()	Return the number of items in the linked list.	int

5.3 Stack

A Stack is a LIFO ADT. Implement it as a wrapper around LinkedList. Use add_first() for push() and remove_first() for pop().

Method	Purpose	Return type
__init__()	Create an empty stack backed by a LinkedList.	None
push(item)	Add an item to the top of the stack.	None
pop()	Remove and return the top item. Return None if empty.	object | None
peek()	Return the top item without removing it. Return None if empty.	object | None
is_empty()	Return True when the stack is empty.	bool
size()	Return the number of items in the stack.	int

5.4 OrderQueue

OrderQueue is your concrete implementation of the Queue ADT. It is FIFO. Use add_last() for enqueue() and remove_first() for dequeue().

Method	Purpose	Return type
__init__()	Create an empty queue backed by a LinkedList.	None
enqueue(item)	Add an item to the rear of the queue.	None
dequeue()	Remove and return the front item. Return None if empty.	object | None
peek()	Return the front item without removing it. Return None if empty.	object | None
is_empty()	Return True when the queue is empty.	bool
size()	Return the number of items in the queue.	int

5.5 ShoppingCart and Store Updates

All Milestone 1 methods remain required. The only new ShoppingCart method is clear(). Store keeps its Milestone 1 product/customer methods and adds the methods below.

Method	Purpose	Return type
ShoppingCart.clear()	Remove all products from the cart.	None
Store.__init__()	Keep M1 products/customers and initialize an empty orders list, an OrderQueue for waiting orders, a processed-order Stack, and an order counter.	None
find_order(order_id: str)	Find and return an Order by ID; return None when not found.	Order | None
get_orders()	Return the store's orders as a list.	list[Order]
checkout(customer_id: str)	Create an Order from the customer's current cart, add it to the store's orders list, enqueue it for processing, clear the cart, and return it. Return None for an unknown customer or empty cart.	Order | None
process_next_order()	Dequeue the oldest waiting order from the OrderQueue, set its status to PROCESSING, push it onto the processed-order Stack, and return the order. Return None if no order is waiting.	Order | None
get_order_history()	Return a list of all orders in the processed-order Stack, from most recently processed to oldest. Do not remove or modify the orders in the Stack.	list[Order]

Why both a list and a queue? The orders list keeps a permanent record of every order the store has received. The OrderQueue contains only orders that are waiting to be processed. The same Order object can appear in both structures, but each structure serves a different purpose.

6. Example Input, Output, and Expected Behavior

The examples below show how the classes should be used. In this project, “input” means method calls and objects passed to your classes; “output” means returned values or observable behavior after the operation. You do not need to use input() for this project.

Example 1: Complete Checkout Workflow

The following sequence shows a customer placing an order. The expected results are shown after the code.

```
# Create the store
store = Store()

# Create products
mouse = Product("P100", "Wireless Mouse", 29.99)
keyboard = Product("P101", "Keyboard", 59.99)
headphones = Product("P102", "Headphones", 39.99)

# Add products to the store
print(store.add_product(mouse))        # Output: True
print(store.add_product(keyboard))     # Output: True
print(store.add_product(headphones))   # Output: True

# Create two customers
alex = Customer("C100", "Alex")
sam = Customer("C101", "Sam")

# Add customers to the store
print(store.add_customer(alex))        # Output: True
print(store.add_customer(sam))         # Output: True

# Alex creates an order
alex_cart = alex.get_cart()
alex_cart.add_product(mouse)
alex_cart.add_product(keyboard)
print(alex_cart.calculate_total())     # Output: 89.98
print(alex_cart.is_empty())            # Output: False
alex_order = store.checkout("C100")

print(alex_order.get_id())             # Output: O1
print(alex_order.get_customer().get_name())   # Output: Alex
print(alex_order.calculate_total())    # Output: 89.98
print(alex_order.get_status())         # Output: PENDING
print(alex_cart.is_empty())            # Output: True
```

```
# Sam creates an order
sam_cart = sam.get_cart()
sam_cart.add_product(headphones)
print(sam_cart.calculate_total())      # Output: 39.99
sam_order = store.checkout("C101")
print(sam_order.get_id())              # Output: O2
print(sam_order.get_customer().get_name())    # Output: Sam
print(sam_order.get_status())          # Output: PENDING

# Process the orders
first_order = store.process_next_order()
print(first_order.get_id())            # Output: O1
print(first_order.get_status())        # Output: PROCESSING
second_order = store.process_next_order()
print(second_order.get_id())           # Output: O2
print(second_order.get_status())       # Output: PROCESSING

# No orders are left in the waiting queue
print(store.process_next_order())      # Output: None

# Check order history
history = store.get_order_history()
print(history[0].get_id())             # Output: O2
print(history[1].get_id())             # Output: O1
```

Use these examples to understand the required inputs, outputs, and observable behavior; your tests should follow the same approach.

These examples are not exhaustive. Your implementation must satisfy all method requirements and edge cases in the specification.

7. Required Testing with unittest

Module 3 introduced Python's built-in unittest framework and the idea of testing behavior, not implementation details. Your team must submit a separate file named test_milestone2.py.

- Import unittest and the classes you are testing.
- Create test classes that inherit from unittest.TestCase.
- Every test method must start with test_.
- Use assertions to check returned values and observable behavior.
- Do not test private implementation details such as _head or _tail. Test what the public methods do.
- Use Red-Green-Refactor as your development process when practical: write a test, make it fail for the right reason, implement the behavior, then refactor.

Test area	Minimum tests	Examples
Order	2	creation/status, total or cart independence
LinkedList	4	add_first, add_last, remove_first, empty/first/size behavior
Stack	3	LIFO order, peek, empty pop
OrderQueue	3	FIFO order, peek, empty dequeue
Store integration	3	checkout, processing, history

Target: at least 15 meaningful tests. The unit-test file is worth 20 points and will be reviewed manually by the TA. Meaningful coverage and correct use of unittest matter more than simply reaching the minimum count.

What to test: Because LinkedList is a required public class in this milestone, test its public behavior directly. Testing only Stack and OrderQueue is not enough. Test public methods and observable results; do not inspect _head, _tail, Node links, or other private implementation details. For Stack and OrderQueue, test their LIFO/FIFO behavior without checking how LinkedList is used internally.

8. Running-Time Analysis

Submit a plain-text file named complexity.txt. For each operation below, provide its worst-case Big-O running time.

- LinkedList.add_first
- LinkedList.add_last
- LinkedList.remove_first
- Stack.push
- Stack.pop
- OrderQueue.enqueue
- OrderQueue.dequeue
- Store.find_product
- Store.find_customer
- Store.find_order

9. Gradescope Submission

Submit all required Milestone 2 files through Gradescope. This milestone is cumulative: your submission should include the working Milestone 1 classes plus the new files and Store updates described here.

File	Status in M2	What it contains
product.py	Same as M1	Product class from Milestone 1.
customer.py	Same as M1	Customer class from Milestone 1.

File	Status in M2	What it contains
cart.py	Updated	M1 behavior plus the new clear() method.
store.py	Updated	M1 behavior plus order, checkout, queue, and history methods.
order.py	New	Order class.
node.py	New	Node class used by LinkedList.
linked_list.py	New	Node-based LinkedList implementation.
stack.py	New	Stack ADT implemented with LinkedList.
order_queue.py	New	Queue ADT implemented with LinkedList.
test_milestone2.py	New	Your team's unittest tests; manually graded.
complexity.txt	New	The 10 required Big-O answers; manually graded.

Gradescope submissions. You may submit as many times as you would like before the deadline. Gradescope will automatically grade the program functionality and provide feedback after each submission. The unittest file and complexity.txt are reviewed manually by the TA. Your final submission before the deadline is used for grading.

10. Code Documentation

Include a short, meaningful docstring for each class and method describing its purpose. Use brief comments when needed to explain important or non-obvious code. Keep comments focused; do not simply restate every line of code.

11. Post-Milestone Feedback & Peer Evaluation

After each milestone, every student must complete an individual contribution and peer-evaluation form describing their own contributions, their teammate's contributions, teamwork and communication, and any resolved or unresolved issues. Although the project is completed as a team, grades may differ based on individual contribution.

12. Grading

Component	Description	Points
Order	Correctly implements the Order class, status behavior, total, and cart-independence.	10
LinkedList	Correctly implements the node-based linked list and required operations.	20
Stack	Correctly implements LIFO behavior using LinkedList.	10
OrderQueue	Correctly implements FIFO behavior using LinkedList.	10
Store integration	Correctly creates, stores, queues, processes, and records orders.	20
Unit tests	Provides the required unittest file with meaningful tests for the new behavior. **Manually graded.**	20
Running-time analysis	Provides the 10 required worst-case Big-O answers in complexity.txt. **Manually graded.**	10
Total		100

Looking Ahead

You now have the same application at two levels: an ADT such as Stack or Queue, and a concrete data structure such as LinkedList. In the next milestones, the store will need faster searching and sorting, followed by larger-scale indexing and more advanced structures. The design decisions you make now will become the foundation for those improvements.

Milestone 2 goal. Make the store correct, testable, and efficient before we ask it to handle much larger collections.